

Project 3. MIPS Pipelined Simulator

Due 23:59, May 19th, 2026 (KST)

TA: Jimin Lee, Dongha Kim (cs30101_ta@casys.kaist.ac.kr)

1. Introduction

This third project is to build a **5-stage pipelined simulator of a subset of the MIPS instruction set**. The simulator loads a MIPS binary into a simulated memory, and executes the instructions. Instruction execution will change the states of registers and memory. The simulator will need to detect and behave accordingly for any data or control hazards. **Please read this document and README.md file provided in the repository carefully before you start.**

****If you have any questions related to the project, please ask them on the Piazza.**

2. GitLab Repository

Basically, you just need to repeat the same procedure of forking and cloning the repository explained in the document for Project 1 and 2. The only difference is that you should start from the CS30101's repository of **Project 3**.

(https://cs30101.kaist.ac.kr/cs30101/project3_mips_pipeline).

Please check prior notifications available in Piazza board, for more information about getting started with GitLab.

3. Required Pipeline Implementation

In this project, you will need to implement the 5 stage MIPS pipeline that you have learned in class. The baseline MIPS simulator (sample solution of project 2) will be given as a backbone.

Pipeline Stages

We will use a simple 5-stage pipeline for this project:

1. IF: fetch a new instruction from memory.
2. ID: decode the fetched instruction and read the register file
3. EX: execute an ALU operation
 - a. Execute arithmetic and logical operations
 - b. Calculate the addresses for loads and stores
4. MEM: access memory for load and store operations
5. WB: write back the result to the register

Between the adjacent pipeline stages, pipeline registers (or pipeline latches) must be modeled. All communication between stages must be conducted using the pipeline registers.

Register File

The register file is used in both the ID stage and the WB stage, however the WB writes the register file at the first half of a cycle and ID reads on the second half of a cycle resulting in no structural hazards in the register file.

Memory Model

We assume an ideal dual-ported memory, which allows both a read at IF, and a read/write at MEM simultaneously within the same cycle. There is no structural hazard for the memory accesses from IF and MEM. You can assume the memory locations where instructions are stored are never updated. Therefore, you do not need to consider the case where a store at MEM updates the same address fetched at IF at the same cycle.

Forwarding

The pipelined architecture must support data forwarding from MEM/WB-to-EX, EX/MEM-to-EX. With the forwarding support, data hazard only occurs for the data dependency from a load to the succeeding instruction which uses the value in the EX stage. MEM/WB-to-MEM is also supported.

Control Hazard

Similar to project 2, we are going to ignore the JAL delay slot for this project. Therefore, when calling the JAL instruction, save the PC+4 value into the R31 (as with project2).

For conditional branches (BEQ, BNE), your simulator must support the static branch predictor which always predicts branches **not taken**. The actual evaluation of the branch will be executed by the ALU and thus the branch result will be ready at the end of the EX stage. However, **the actual flushing will take place at the beginning of the MEM stage**. A correct branch (not taken) will not incur any stalls; a miss prediction will cause 3 cycle stall

(flushing of the EX, IF, and ID of the newer instructions). Also, if the branch decision has dependencies on prior executions, stall the execution at the appropriate stage.

Stopping the Pipeline

The simulator must stop after a given number of instructions finishes the WB stage. At cycle 1, the first instruction is fetched from the memory. If the last instruction is in the WB stage at cycle N, the final CYCLE count is N.

Pipeline Register States

You need to add pipeline register states between stages. The followings are possible register contents, but you need to add more states.

IF_ID.Instr : 32-bit instruction
IF_ID.NPC : 32-bit next PC (PC+4)
ID_EX.NPC : 32-bit next PC
ID_EX.REG1 : REG1 value
ID_EX.REG2 : REG2 value
ID_EX.IMM : Immediate value
EX_MEM.ALU_OUT: ALU output
EX_MEM.BR_TARGET: Branch target address
MEM_WB.ALU_OUT: ALU output
MEM_WB.MEM_OUT: memory output

You must carefully design what fields are necessary for each pipeline register, and explain them in "README" file in your submission. You must explain what each field means.

For more information about simulator requirements, please see project 2 document.

4. Simulator Options

Basic command

\$./cs311sim [-m addr1:addr2] [-d] [-n num_instr] <input file>

Options

- m : Print the memory content from addr1 to addr2
- d : Print the register file content for each instruction execution. Print the memory content together if -m option is specified.
- n : number of instructions simulated
- p : Print the PCs of the instructions in each pipeline stage at every cycle. Prints a blank if the stage of the pipeline is stalled or empty.

5. Grading Policy

Basically, grading policy is just same as project 2. Submissions will be graded based on the 10 examples: **6 given examples in 'sample_input' directory, and 4 hidden examples which inputs and outputs are both hidden.** Hidden examples have similar complexity with respect to given examples.

To get score, your simulator should print the exact same output as the reference solution. You can evaluate your code within given examples by comparing your output with files in 'sample_output' directory. **You may check the correctness of your code by executing make test at your working directory of this project.**

If there are any differences (including whitespaces) it will print the differences. If there is no difference for an example, it will print "Test seems correct".

Please make sure your outputs for given examples are **identical to the files in the sample_output directory, without any redundant prints. Every single character of the output must be identical** to the given sample output. Otherwise, you will receive **0 score** for the example.

Note that **simply hard-coding outputs for given examples would lead you to 0 score** for this project.

6. Submission (**Important!!**)

Make sure your code works well on our class Linux server.

It is highly recommended to work on the class server, since your project will be graded on the same environment as those servers.

Add the 'submit' tag to your final commit and push your work to the gitlab server.

The following commands are the flow you should take to submit your work.

If there is no “submit” tag, your work will not be graded. Please do not forget to submit your work with the tag.

Please make sure you push your work before the deadline. If you do not “push” your work, we won't be able to see your work so that your work will be marked as unsubmitted.

For more information about submission procedure, please refer to the specification of project 1, chapter 5 ‘Submission’.

7. Late Policy & Plagiarism Penalty (**Important!!**)

You will lose **30%** of your score on the **first day** (May 20th 0:00~23:59). We will **not accept** any works that are submitted after then.

Be aware of plagiarism! Although it is encouraged to discuss with others and refer to extra materials, **copying other students' or opened code is strictly banned: Not only for main routine functions, but also helper functions.**

TAs will compare your source code with open-source codes and other students' code.

If you are caught, you will receive a serious penalty for plagiarism as announced in the first lecture of this course.

If you have any requests or questions regarding administrative issues (such as late submission due to an unfortunate accident, GitLab is not working) please send an e-mail to TAs.

8. Tips

Modifying your repository via http may cause some problems.

You may use C++ instead of C if you want. Just make sure your 'make test' command works properly.

Please, do not modify contents (workflows and functionalities) of files other than util.h, run.h, run.c, and Makefile.

Please start as early as possible.

Please read this document, README.md, and given skeleton code carefully before you start.